WBS TRAINING

Handout

TF-US-PSS

Algorithmus Theorie & Pseudocode

WBS TRAINING AG
Lorenzweg 5
D-12099 Berlin
Amtsgericht Berlin HRB 68531
Sitz der Gesellschaft: Berlin

Vorstand:
Heinrich Kronbichler,
Joachim Giese
Aufsichtsrat (Vorsitz): Dr. Daniel Stadler
USt-IdNr.: DE 209 768 248

GLS Gemeinschaftsbank eG
IBAN: DE18 4306 0967 1146 1814 00
BIC: GENODEM1GLS

DQS-zertifiziert nach
DIN EN ISO 9001:2015 Reg.-Nr. 015344 QM15
Zulassung nach AZAV Reg.-Nr. 015344 AZAV

Inhalt

1 Algorithmus 3
1.1 Definition 3
1.2 Die zwingenden Haupteigenschaften eines Algorithmus 3
1.3 Vergleich: Das Kochrezept 3
1.4 Weitere Beispiele 4
1.5 Merke: 4
2 Pseudo-Code 4
2.1 Definition 4
2.2 Beispiele für Grundlegende Anweisungen im Pseudocode 5
2.3 Beispiele für einen Algorithmus in Pseudocode 5
2.4 Programmieren versus Codieren 6
2.5 Variablen 6
2.5.1 EDV = Elektronische Daten-Verarbeitung 6
2.5.2 Variablen – Zuweisung 7
2.5.3 Zuweisung von Werten einer anderen Variable 8
2.5.4 Zuweisungen eines Rechenergebnisses 9
2.5.5 Zuweisung von reflexiven Ausdrücken 9
2.6 Kontrollstrukturen 10
2.6.1 Sequenz (Reihenfolge) 10
2.6.2 Selektion (Auswahl/Entscheidung) 10
2.6.3 Iteration (Wiederholung/Schleife) 11

1 Algorithmus

1.1 Definition

Ein Algorithmus ist eine eindeutige Handlungsvorschrift, die ein klar bestimmtes Problem auf eine konkrete Art löst. Er besteht aus einer geordneten Menge von Handlungsschritten, welche die Lösungs-Idee darstellen und nicht von einer bestimmten Technologie abhängig sind (z.B. Programmiersprache).

1.2 Die zwingenden Haupteigenschaften eines Algorithmus

Eigenschaft	Erklärung	Fachbegriff(e)
Eindeutigkeit	Die Anweisungen sind klar, präzise und lassen zu keinem Zeitpunkt Raum für Interpretationen. Es ist immer klar, was der nächste Schritt ist.	Determinismus
Endlichkeit	Der Algorithmus selbst muss eine begrenzte Beschreibungslänge haben (statische Finitheit) und die Ausführung muss nach einer begrenzten Anzahl von Schritten ein Ergebnis liefern oder anhalten (Terminierung/dynamische Finitheit).	Finitheit & Terminierung
Ausführbarkeit	Jeder einzelne Schritt muss tatsächlich vom Prozessor (oder dem Ausführenden) machbar sein (elementare Operationen). Eine Division durch Null wäre zum Beispiel nicht ausführbar.	Effektivität
Ein- und Ausgabe	Er muss definierte Eingabewerte (Daten) entgegennehmen und ein definiertes Ergebnis (Ausgabe) erzeugen.	Input/Output
Allgemeinheit	Ein Algorithmus soll eine ganze Klasse von Problemen der gleichen Art lösen und nicht nur einen Einzelfall (z. B. eine allgemeine Formel zum Lösen aller quadratischen Gleichungen).	Generalität

1.3 Vergleich: Das Kochrezept

Ein Kochrezept ist ein perfektes Alltagsbeispiel für einen Algorithmus: Es liefert eine klare Schritt-für-Schritt-Anleitung (Anweisungsabfolge), die zu einem bestimmten Ergebnis (dem fertigen Gericht) führt, vorausgesetzt alle Schritte sind machbar und die Menge der Schritte ist begrenzt.

1.4 Weitere Beispiele

Darstellungen von geordneten Handlungsvorschriften finden sich nicht nur in der Programmierung:

- Bauanleitungen
- Anleitungen zum Aufbau eines Brettspiels
- Schritt-für-Schritt-Tutorial Videos
- Suchalgorithmen
- Sortieralgorithmen
- Kompressionsalgorithmen
- Verschlüsselungsalgorithmen

1.5 Merke:

Ein Algorithmus ist die **Logik** hinter dem Programm. Das Programm ist lediglich die konkrete Umsetzung dieser Logik in einer bestimmten Programmiersprache.

2 Pseudo-Code

Bevor man ein Problem in einer spezifischen Programmiersprache implementiert, plant man die Logik oft in **Pseudocode**.

2.1 Definition

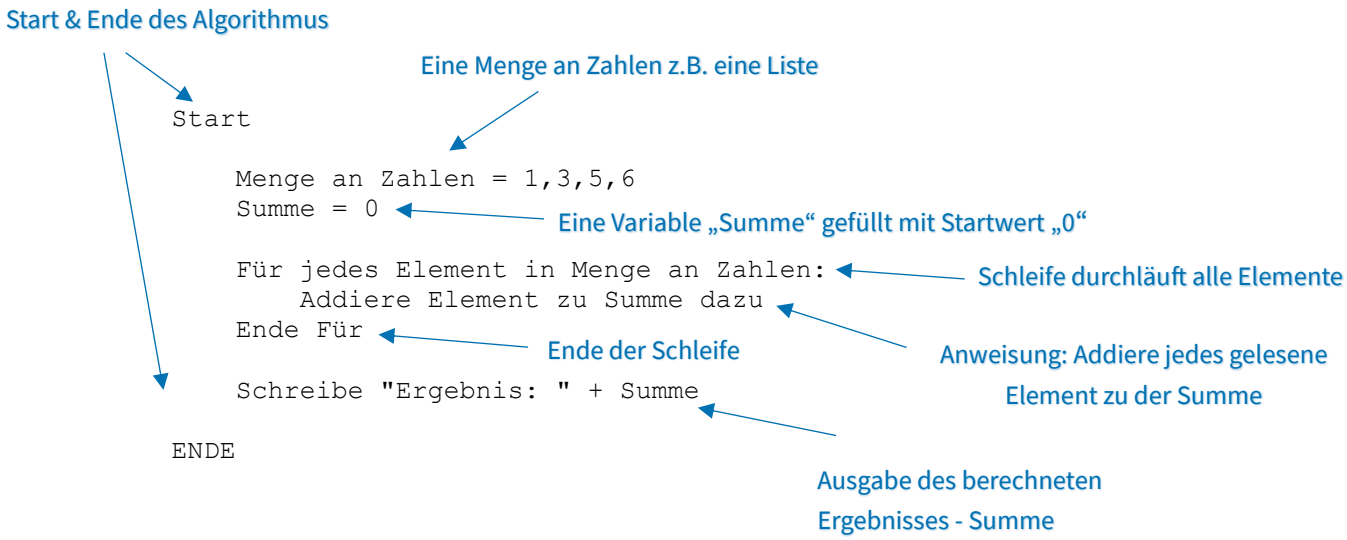
„Pseudo“ steht für „nicht real“ oder „so ähnlich wie, aber nicht das Richtige“. Pseudo-Code ist eine Modellierungsform und keine Programmiersprache. Damit kann man Sprachübergreifend Ideen formulieren oder Algorithmen darstellen. In der Praxis sieht man oft, dass doch einfach die Syntax einer existierenden Sprache verwendet wird, solange die einzelnen Anweisungen für Programmierer allgemein verständlich sind (also ohne sehr spezielle Spracheigenschaften zu verwenden). Aber auch normale Sätze und Teilsätze können korrekten Pseudo-Code abbilden.

2.2 Beispiele für Grundlegende Anweisungen im Pseudocode

Die Basisbefehle ähneln oft einfachen Verben oder Schlüsselwörtern:

Anweisung	Beispiel	Erläuterung
Beginn/Ende	`START,ENDE`	Kennzeichnen den gesamten Algorithmus.
Zuweisung	`SETZE Summe = 0` oder `Summe := 0`	Weist einer Variablen einen Wert zu (Initialisierung, Berechnung).
Eingabe	`LIES Zahl`	Liest einen Wert von der Eingabe (z. B. Tastatur) und speichert ihn in der Variablen Zahl.
Ausgabe	`SCHREIBE "Ergebnis: " + Durchschnitt`	Gibt einen Text und/oder den Wert einer Variablen aus.
Verarbeitung	`Ergebnis = a * 10`	Führt eine Berechnung durch.

2.3 Beispiele für einen Algorithmus in Pseudocode



2.4 Programmieren versus Codieren

Der Unterschied zwischen **Programmieren** und **Codieren** wird oft synonym verwendet, aber in einem präziseren Kontext der Softwareentwicklung beschreibt **Programmieren** einen umfassenderen Prozess, während **Codieren** den spezifischen Akt des Schreibens von Code meint.

„**Programmieren**“ meint das **Planen** sowie *(graphische, und/oder sprachliche)* **Beschreiben** aller Arbeitsschritte, die zur Lösung eines gegebenen Problems abzuarbeiten sind.

„**Codieren**“ hingegen meint das **Übersetzen** dieses graphischen Plans … in eine für den Computer (oder Compiler / Interpreter) verständliche Sprache ... sowie das **Implementieren** (Eintragen) dieser Übersetzung in den Rechner.

2.5 Variablen

2.5.1 EDV = Elektronische Daten-Verarbeitung

Die elektronische Datenverarbeitung (**EDV**) basiert auf einem einfachen Prinzip: Programme verarbeiten **Daten**. Dazu müssen die Daten erst einmal **gespeichert** werden.

- **Die Rolle der Variablen**

Die Speicherung von Daten kann aus zwei Perspektiven betrachtet werden:

- **Technisch:** Auf der Hardware-Ebene werden die Daten in den **Speicherzellen** des Computers abgelegt.
- **Für den Programmierer:** Diese Speicherplätze werden abstrakt als **Variablen** bezeichnet.
- **Adressierung und Benennung**

Um auf gespeicherte Daten zuzugreifen, müssen die Speicherorte eindeutig identifizierbar sein:

- **Technisch** erfolgt dies über fortlaufende, eindeutige **Adressen** (Zählerwerte), mit denen die Speicherzellen versehen sind.
- **Der Programmierer** nutzt stattdessen **selbstgewählte Namen** für seine Variablen, die als intuitive Referenz für die eigentlichen Speicheradressen dienen.
- **Warum "Variable"?**

Der Begriff **Variable** (wörtlich: *Veränderliche*) weist darauf hin, dass der in ihr gespeicherte Wert **während der Programmausführung gezielt geändert** werden kann. Diese geplante Flexibilität ist vom Programmierer bewusst vorgesehen und für dynamische Programme unerlässlich.

Übersicht:

Aspekt	Sicht des Computers (Technisch)	Sicht des Programmierers (Abstrakt)
Speicherort	**Speicherzellen** (physischer Platz im RAM)	**Variable** (ein benannter Behälter für einen Wert)
Identifikation	**Adressen** (eindeutige Zählerwerte)	**Variablen-Namen** (selbstgewählter Bezeichner)
Kernfunktion	Der Wert in den Zellen kann sich im Laufe der Zeit **ändern**.	Die **Variable** ermöglicht die **gezielte Veränderung** des Wertes während der Laufzeit.

Wichtiger Hinweise für diesen Kurs (PSS)

Fokus auf den Pseudocode

In diesem Kurs (**PSS**) konzentrieren wir uns auf die **logischen Abläufe** und die **grundlegenden Konzepte** der Programmierung, die wir im **Pseudocode** darstellen.

Vereinfachte Variablen-Verwendung

Im Pseudocode werden wir uns vorerst **nicht mit den spezifischen Datentypen** beschäftigen. Das heißt:

- Wir verwenden eine Variable (z.B. *Zahl*, *Text*, *Wahr/Falsch*) und **müssen nicht ausdrücklich festlegen**, ob sie eine Ganzzahl (Integer) oder eine Kommazahl (Float) ist.
- **Der Mensch kann sich den Typ logisch erschließen.**

Die **detaillierte Zuweisung und Unterscheidung** der Datentypen (Integer, Float, Character, etc.) wird ein zentrales Thema im nächsten Kurs **PPP** sein, da diese Unterscheidung für den *eigentlichen Computer* beim Codieren zwingend notwendig ist, um den korrekten Speicherplatz zu reservieren.

2.5.2 Variablen – Zuweisung

Ähnlich wie in einer Programmiersprache muss einer **Variable** im Pseudocode ein Wert zugewiesen werden. Dies geschieht durch den **Zuweisungsoperator**, der üblicherweise als **=** oder, in manchen Konventionen, als **:=** dargestellt wird. Obwohl beide Symbole im Pseudocode verwendet werden können, ist es wichtig, sich für eine Schreibweise zu entscheiden und diese **durchgängig** beizubehalten.

Beispiel:

Die Schreibweise **a=5** bedeutet, dass der Variable **a** nun der (ganzzahlige) Wert **5** zugewiesen wurde

Die Schreibweise **preis=9.95** bedeutet, dass der Variable **preis** nun der (Komma)-Wert **9.95** zugewiesen wurde

Die Schreibweise **meinLieblingsZeichen=‘q‘** bedeutet, dass der Variable **meinLieblings-Zeichen** nun der (Character)-Wert **‘q‘** zugewiesen wurde

2.5.3 Zuweisung von Werten einer anderen Variable

Wir können einer Variablen auch den Wert einer **anderen Variable** (vom selben Typ) zuweisen. Dies macht nur Sinn, wenn die zugewiesene Variable bereits einen **definierten** Wert besitzt.

Beispiele:

Angenommen der Variable **a** wurde der Wert **22** zugewiesen:

a=22

dann bedeutet die Schreibweise …

b=a

dass der Variablen **b** der Wert von **a** (in diesem Fall also 22) zugewiesen wurde

=> a und b speichern nun also **denselben** Wert.

Angenommen der Variable **zeichen1** wurde der Wert **‘x‘** zugewiesen:

zeichen1=‘x‘

dann bedeutet die Schreibweise …

zeichen2=zeichen1

dass der Variablen **zeichen2** der Wert von **zeichen1** (in diesem Fall also ‘x‘) zugewiesen wurde

=> zeichen1 und zeichen2 speichern nun also **denselben** Wert.

2.5.4 Zuweisungen eines Rechenergebnisses

Wir können einer (Zahlen)-Variablen auch ein **Rechenergebnis** zuweisen.

Beispiel:

a=5

b=a+1 (b hat nun also den Wert 6)

c=a-3 (c hat nun also den Wert 2)

d=2*a (d hat nun also den Wert 10)

e=a/2 (e hat nun also den Wert 2)

weitere Beispiele:

a=3

b=a+2 (b hat nun also den Wert 5)

c=a-b (c hat nun also den Wert -2)

d=a*a (d hat nun also den Wert 9)

e=b/d (e hat nun also den Wert 0)

2.5.5 Zuweisung von reflexiven Ausdrücken

a=1

a=a+1 (a hat nun also den Wert 2)

a=a-10 (a hat nun also den Wert -8)

a=(-2)*a (a hat nun also den Wert 16)

a=a/a (a hat nun also den Wert 1)

2.6 Kontrollstrukturen

Jeder denkbare Algorithmus – egal wie komplex – lässt sich nach dem Prinzip der **Strukturierten Programmierung** ausschließlich aus den folgenden drei fundamentalen Kontrollstrukturen zusammensetzen:

2.6.1 Sequenz (Reihenfolge)

Prinzip: Anweisungen werden in der festgelegten Reihenfolge, eine nach der anderen, ausgeführt.

Wichtig: Ein Schritt wird erst begonnen, wenn der vorherige vollständig abgeschlossen ist.

Pseudocode-Beispiel:

```
START
SETZE a = 10
SETZE b = 20
SETZE Summe = a + b
SCHREIBE Summe
ENDE
```

2.6.2 Selektion (Auswahl/Entscheidung)

Prinzip: Ermöglicht die Entscheidungsfindung im Algorithmus mithilfe einer Bedingung (die entweder WAHR oder FALSCH ist).

Struktur: Führt je nach Ergebnis der Bedingung unterschiedliche Codeblöcke aus (Verzweigung).

Pseudocode-Syntax:

```
WENN [Bedingung] DANN
    [Anweisungen_bei_WAHR]
SONST
    [Anweisungen_bei_FALSCH]
ENDE WENN
```

Pseudocode-Beispiel:

```
WENN Alter >= 18 DANN
    SCHREIBE "Zugang erlaubt"
SONST
    SCHREIBE "Zugang verweigert"
ENDE WENN
```

2.6.3 Iteration (Wiederholung/Schleife)

Prinzip: Ermöglicht das wiederholte Ausführen eines Codeblocks (Schleifenrumpf), solange eine bestimmte Bedingung erfüllt ist oder bis eine Bedingung eintritt.

Drei Hauptformen:

Kopfgesteuert (Prüfung am Anfang): Der Rumpf wird nur ausgeführt, wenn die Bedingung *vor* dem ersten Durchlauf WAHR ist.

```
SOLANGE [Bedingung] WIEDERHOLE
    [Anweisungen]
ENDE SOLANGE
```

Beispiel:

```
START
    SETZE i = 1
    SOLANGE i <= 3 WIEDERHOLE
        SCHREIBE i // Ausgabe: 1, dann 2, dann 3
        SETZE i = i + 1
    ENDE SOLANGE
ENDE
```

Fußgesteuert (Prüfung am Ende): Der Rumpf wird garantiert **mindestens einmal** ausgeführt, da die Prüfung erst nach dem ersten Durchlauf erfolgt.

```
WIEDERHOLE
    [Anweisungen]
BIS [Bedingung]
```

Beispiel:

```
START
    SETZE i = 1
    WIEDERHOLE
        SCHREIBE i // Ausgabe: 1, dann 2, dann 3
        SETZE i = i + 1
    BIS i > 3 // Die Bedingung ist die ABBRUCH-Bedingung
ENDE
```

Zählergesteuert (Steuerung durch Zählvariable):

Prinzip: Die Anzahl der Wiederholungen ist **bekannt** und fest definiert. Eine Zählvariable (**Laufindex**) wird automatisch von einem **Startwert** bis zu einem **Endwert** hoch- oder heruntergezählt. Der Codeblock wird exakt **n-mal** ausgeführt.

Ergänzung zur Ausführung: Der Rumpf wird **für jeden Schritt des Zählers** innerhalb des definierten Bereichs einmal ausgeführt.

Zähle hoch: vom Startwert bis zum Endwert, Schrittweite 1 (implizit)

```
FÜR Zaehler VON [Startwert] BIS [Endwert] WIEDERHOLE
    [Anweisungen]
ENDE FÜR
```

Zähle hoch mit spezifischer Schrittweite

```
FÜR Zaehler VON [Startwert] BIS [Endwert] SCHRITT [Schrittweite] WIEDERHOLE
    [Anweisungen]
ENDE FÜR
```

Zähle runter (Dekrement)

```
FÜR Zaehler VON [Startwert] ABWÄRTS BIS [Endwert] WIEDERHOLE
    [Anweisungen]
ENDE FÜR
```

Beispiel:

```
START
    // FÜHRT den Code 5-mal aus: i beginnt bei 1, endet bei 5
    FÜR i VON 1 BIS 5 WIEDERHOLE
        SETZE Ergebnis = i * 7
        SCHREIBE i + " mal 7 ist gleich " + Ergebnis
    ENDE FÜR

    // Ausgabe:
    // 1 mal 7 ist gleich 7
    // 2 mal 7 ist gleich 14
    // 3 mal 7 ist gleich 21
    // 4 mal 7 ist gleich 28
    // 5 mal 7 ist gleich 35
ENDE
```